Algoritmia

Práctica 3

Javier Ortín Rodenas

8/3/2026

Contenidos

1. Divide y vencerás por sustracción 2
   - 1.1. Complejidades teóricas de los algoritmos 2
   - 1.2. Análisis empírico de las complejidades 3
   - 1.3. Valores de n para los que se aborta la ejecución 4
   - 1.4. Final de `sustraccion3` para tamaño $n = 80$ 4
   - 1.5. Toma de tiempos para `sustraccion4` 5
   - 1.6. Toma de tiempos para `sustraccion5` 5
   - 1.7. Tiempo para `sustraccion5` con $n = 80$ 6
2. Divide y vencerás por división 7
   - 2.1. Complejidades teóricas 7
   - 2.2. Toma de tiempos para `division4` 8
   - 2.3. Toma de tiempos para `division5` 9
3. Métodos concretos 11
   - 3.1. Tiempos empíricos de Fibonacci 11
   - 3.2. Gráficas de `Fibonacci` 12
   - 3.3. Tiempos de `VectorSuma` 14
4. Puntos más cercanos de un conjunto 16
   - 4.1. Ejecución secuencial 16
   - 4.2. Paralelización del algoritmo 17
   - 4.3. Cálculo del umbral 18
   - 4.4. Comparación con la versión secuencial 21
5. Condiciones de medida 24
   - 5.1. Condiciones para la práctica 24

§1. Divide y vencerás por sustracción

1.1. Complejidades teóricas de los algoritmos

Usaremos el Teorema Maestro para calcular la complejidad temporal para cada método recursivo de esta práctica. Siguiendo la notación usual,

- **Sustracción 1:**

$$\begin{cases} k = 0 \\ a = 1 \\ b = 1 \end{cases} \Rightarrow T_{S_1}(n) = O(n)$$

- **Sustracción 2:**

$$\begin{cases} k = 1 \\ a = 1 \\ b = 1 \end{cases} \Rightarrow T_{S_2}(n) = O(n^2)$$

- **Sustracción 3:**

$$\begin{cases} k = 0 \\ a = 2 \\ b = 1 \end{cases} \Rightarrow T_{S_3}(n) = O\left(2^n\right)$$

- **Sustracción 4:**

$$\begin{cases} k = 2 \\ a = 1 \\ b = 1 \end{cases} \Rightarrow T_{S_4}(n) = O(n^3)$$

- **Sustracción 5:**

$$\begin{cases} k = 0 \\ a = 3 \\ b = 2 \end{cases} \Rightarrow T_{S_5}(n) = O\left(3^{\frac{n}{2}}\right)$$

1.2. Análisis empírico de las complejidades

Sin hacer tabla de tiempos, vemos si las mediciones respaldan los tiempos empíricos de los tres primeros algoritmos de sustracción.

- **Sustracción 1:** $O(n)$ Analizando tiempos totales (ms) a 10.000 repeticiones:

  i) $T(1.000) = 217$

  ii) $T(8.000) = 1.646$

  $$\frac{8.000}{T(8.000)} = 4,8603 \sim 4,6083 = \frac{1.000}{T(1.000)}$$

  El cociente de la función de complejidad esperada y los tiempos empíricos es similar en ambos casos. Por tanto, los datos empíricos respaldan el resultado teórico esperable.

- **Sustracción 2:** $O(n^2)$ Comparando tiempos totales (ms) a 10 repeticiones:

  i) $T(1.000) = 66$

  ii) $T(8.000) = 4.017$

  $$\frac{1.000^2}{T(1.000)} = 15.151,5151 \sim 15.932,2878 = \frac{8.000^2}{T(8.000)}$$

- **Sustracción 3:** $O(2^n)$ Comparando tiempos totales (ms) a una repetición:

  i) $T(24) = 683$

  ii) $T(29) = 22.057$

  $$\frac{2^24}{T(24)} = 24.564,0059 \sim 24.340,1601 = \frac{2^29}{T(29)}$$

1.3. Valores de n para los que se aborta la ejecución

Ejecutando las clases `Sustraccion1` y `Sustraccion2` (con una sola repetición en cada caso), podemos observar que se aborta la ejecución:

- **Sustracción 1:**

```
n=1000 TIEMPO=0  nVeces=1
n=2000 TIEMPO=0  nVeces=1
n=4000 TIEMPO=0  nVeces=1
n=8000 TIEMPO=1  nVeces=1
Exception in thread "main" java.lang.StackOverflowError
```

- **Sustracción 2:**

```
n=1000 TIEMPO=6  nVeces=1
n=2000 TIEMPO=27  nVeces=1
n=4000 TIEMPO=104  nVeces=1
n=8000 TIEMPO=400  nVeces=1
Exception in thread "main" java.lang.StackOverflowError
```

Para los dos métodos, la altura del árbol de llamadas es tal que se llega a un desbordamiento de pila. En ambos casos, cada llamada al método correspondiente con tamaño n induce otra de tamaño $n-1$ hasta que el problema tenga un tamaño fijo (caso base). Así, la altura del árbol de llamadas es lineal: $O(n)$.

1.4. Final de `sustraccion3` para tamaño $n = 80$

Basándonos en los datos empíricos medidos para este método, para el caso $n = 29$ tenemos un tiempo de ejecución de $22.057ms$. Comparando ahora con la complejidad teórica:

$$t_2 = t_1 \cdot \frac{f(n_2)}{f(n_1)} = 22.057ms \cdot \frac{2^{80}}{2^{29}} \cdot \frac{1h}{3,6 \cdot 10^6} ms \cdot \frac{1\text{año}}{24 * 365h} = 1,5750 \cdot 10^9 \text{años}$$

1.5. Toma de tiempos para `sustraccion4`

Tal y como hemos mencionado en el apartado 1.1, para lograr la complejidad teórica de $O(n^3)$, hemos empleado recursividad por sustracción haciendo uso de los siguientes parámetros:

$$k = 2 \qquad a = 1 \qquad b = 1$$

Veamos qué tiempos empíricos registramos para este algoritmo:

n	Tiempo (ms)	Repeticiones	n^3/T_{medio}
100	51	10	196.078,4314
200	384	10	208.333,3333
400	2.831	10	226.068,5270
800	22.475	10	227.808,6763
1.600	17.705	1	231.347,0771
3.200	142.593	1	229.800,9019
6.400	FdT	-	-

Podemos observar que los tiempos empíricos respaldan la complejidad teórica.

1.6. Toma de tiempos para `sustraccion5`

Análogamente, para este método:

n	Tiempo (ms)	Repeticiones	3ˆ{n/2}/T
30	422	1	34.002,1493
32	1.280	1	33.630,2508
34	3.792	1	34.055,9502
36	11.445	1	33.850,6325
38	34.310	1	33.875,2978
40	103.475	1	33.696,8775
42	FdT	-	-

Sigue la complejidad esperada

1.7. Tiempo para `sustraccion5` con $n = 80$

Razonando como en el apartado 1.4, al comparar con la tabla anterior,

$$t_2 = 103.475ms \cdot \frac{3^{\frac{80}{2}}}{3^{\frac{40}{2}}} \cdot \frac{1h}{3,6 \cdot 10^6} ms \cdot \frac{1\text{año}}{24 * 365h} = 1,144 \cdot 10^4 \text{años}$$

§2. Divide y vencerás por división

2.1. Complejidades teóricas

- **División 1:**

$$\begin{cases} k = 1 \\ a = 1 \\ b = 3 \end{cases} \Rightarrow T_{D_1}(n) = O(n)$$

- **División 2:**

$$\begin{cases} k = 1 \\ a = 2 \\ b = 2 \end{cases} \Rightarrow T_{D_2}(n) = O(n \log n)$$

- **División 3:**

$$\begin{cases} k = 0 \\ a = 2 \\ b = 2 \end{cases} \Rightarrow T_{D_3}(n) = O(n)$$

- **División 4:**

$$\begin{cases} k = 2 \\ a = 1 \\ b = 2 \end{cases} \Rightarrow T_{D_4}(n) = O(n^2)$$

- **División 5:**

$$\begin{cases} k = 0 \\ a = 4 \\ b = 2 \end{cases} \Rightarrow T_{D_5}(n) = O(n^2)$$

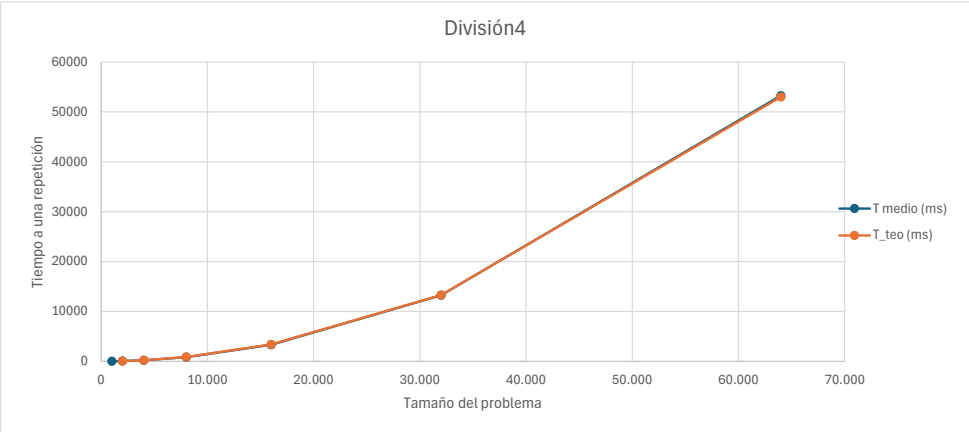
Todas las complejidades han sido halladas usando el Teorema Maestro visto en teoría. Para los dos últimos casos, hemos obtenido los parámetros a partir de las complejidades teóricas buscadas.

2.2. Toma de tiempos para `division4`

Hemos medido los siguientes tiempos para el cuarto algoritmo de DyV por división:

n	$T_{\text{total}}(ms)$	Repeticiones	n^2/T_{medio}	T_{teo} (ms)
1.000	133	10	75.187,9699	-
2.000	52	1	76.923,0769	53,2
4.000	219	1	73.059,3607	208,0
8.000	845	1	75.739,6450	876,0
16.000	3.319	1	77.131,6662	3.380,0
32.000	13.251	1	77.277,1866	13.276,0
64.000	53.268	1	76.894,1954	53.004,0
128.000	FdT	-	-	-

Los resultados empíricos respaldan las complejidades teóricas

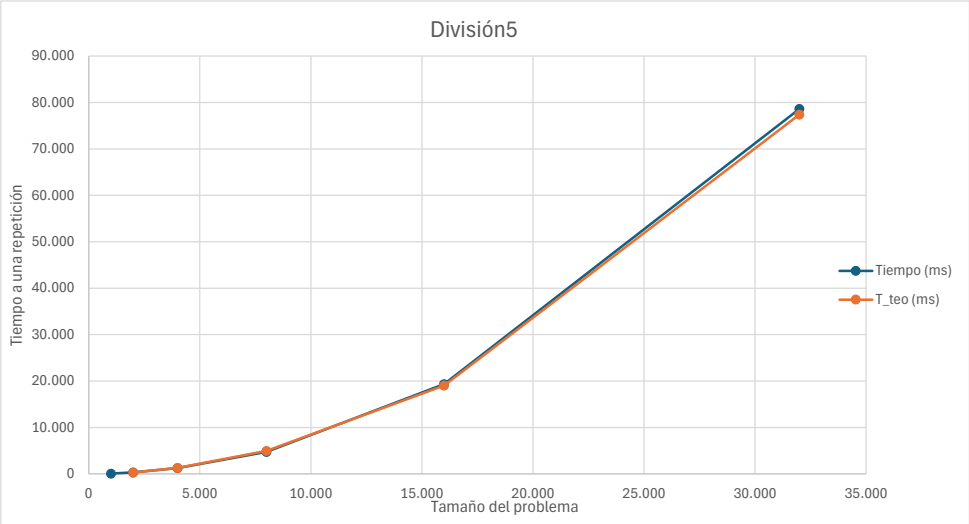


Los tiempos promedios empíricos y los tiempos teóricos a una repetición son casi idénticos. Hemos usado la complejidad $O(n^2)$ para los cálculos de tiempos teóricos.

2.3. Toma de tiempos para `division5`

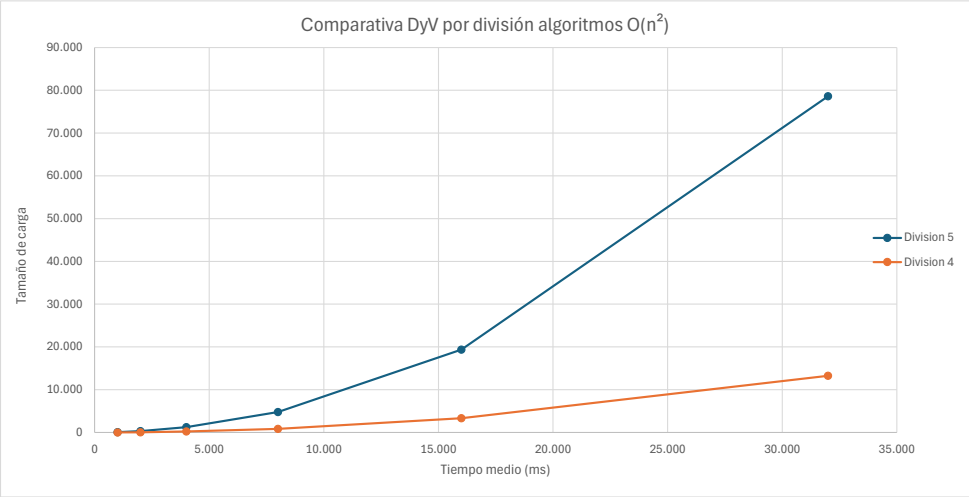
De nuevo, los resultados obtenidos son los siguientes:

n	Tiempo (ms)	Repeticiones	n^2/T	T_{teo} (ms)
1.000	74	1	13.513,5135	-
2.000	322	1	12.422,3602	296
4.000	1.240	1	12.903,2258	1.288
8.000	4.760	1	13.445,3782	4.960
16.000	19.352	1	13.228,6069	19.040
32.000	78.616	1	13.025,3384	77.408
64.000	FdT	-	-	-



Los tiempos promedios empíricos y los tiempos teóricos a una repetición son casi idénticos. Hemos usado la complejidad $O(n^2)$ para los cálculos de tiempos teóricos.

Ambos algoritmos tienen la misma complejidad teórica, pero los tiempos registrados están lejos entre sí, aun habiendo medido en la misma máquina. Veamos una comparación gráfica entre los tiempos medidos:



Podemos observar una clara diferencia entre los algoritmos.

§3. Métodos concretos

3.1. Tiempos empíricos de Fibonacci

Estos han sido los tiempos empíricos para cada método, junto con la estimación teórica:

n	T_{total} (ms)	Repeticiones	n/T_{medio}	T_{teo} (ms)
40	651	1.000.000	61.443,9324	-
41	661	1.000.000	62.027,2315	667,2750
42	673	1.000.000	62.407,1322	677,1220
43	692	1.000.000	62.138,7283	689,0238

Método 1

n	T_{total} (ms)	Repeticiones	n/T_{medio}	T_{teo} (ms)
40	909	1.000.000	44.004,4004	-
41	933	1.000.000	43.944,2658	931,7250
42	948	1.000.000	44.303,7975	955,7561
43	991	1.000.000	43.390,5146	970,5714

Método 2

n	T_{total} (ms)	Repeticiones	n/T_{medio}	T_{teo} (ms)
40	1.790	1.000.000	22.346,3687	-
41	1.722	1.000.000	23.809,5238	1.834,7500
42	1.726	1.000.000	24.333,7196	1.764,0000
43	1.750	1.000.000	24.571,4286	1.767,0952

Método 3

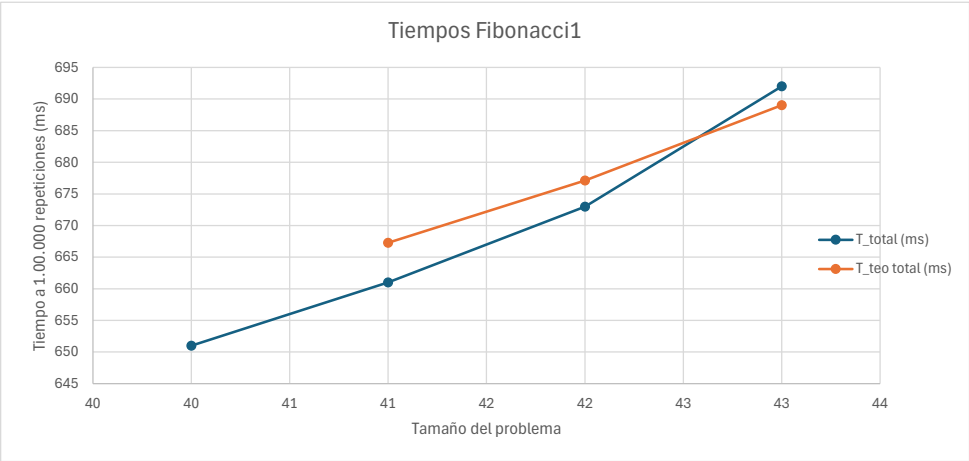
n	T_{total} (ms)	Repeticiones	$1.6^n/T_{\text{medio}}$	T_{teo} (ms)
40	11.260	1.000.000	12.979.588.253,3829	-
41	18.217	1.000.000	12.836.376.020,9115	18.016,0000
42	30.258	1.000.000	12.365.140.430,8518	29.147,2000
43	50.022	1.000.000	11.967.355.776,4732	48.412,8000

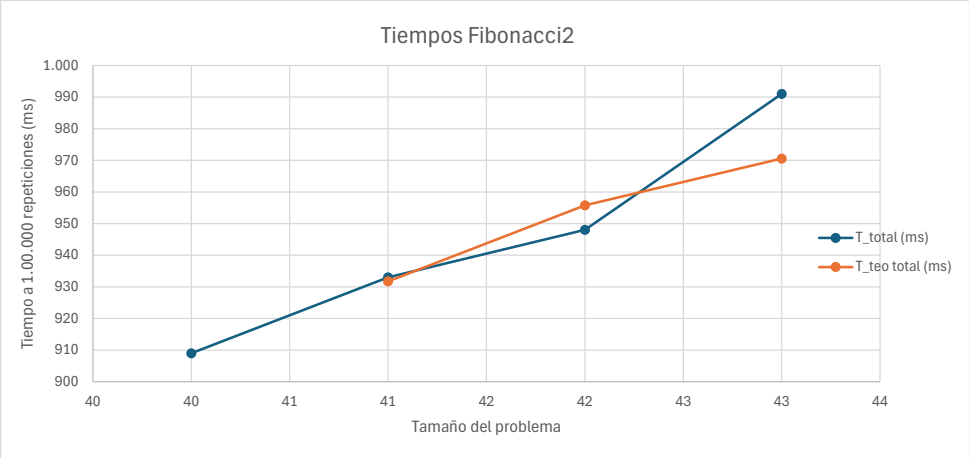
Método 4

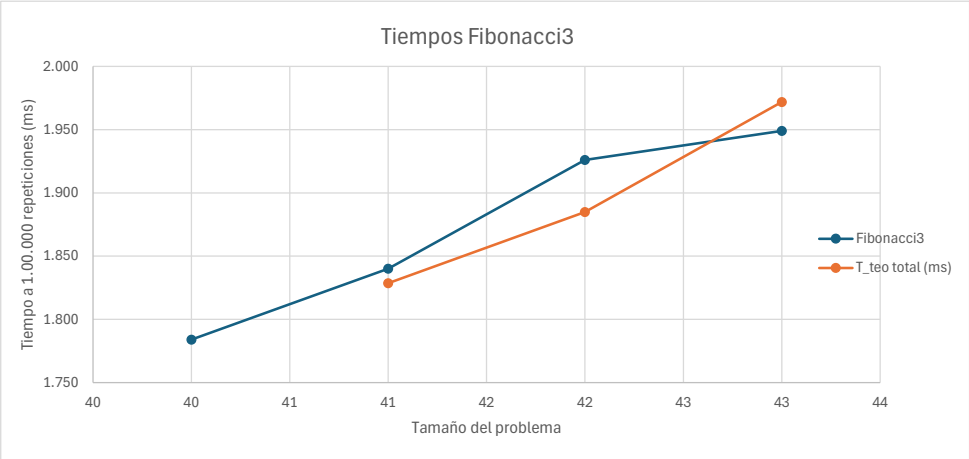
Podemos observar cómo, aun teniendo la misma complejidad teórica, los algoritmos presentan tiempos empíricos diferentes. En particular, el cuarto algoritmo tiene una complejidad muy superior al resto.

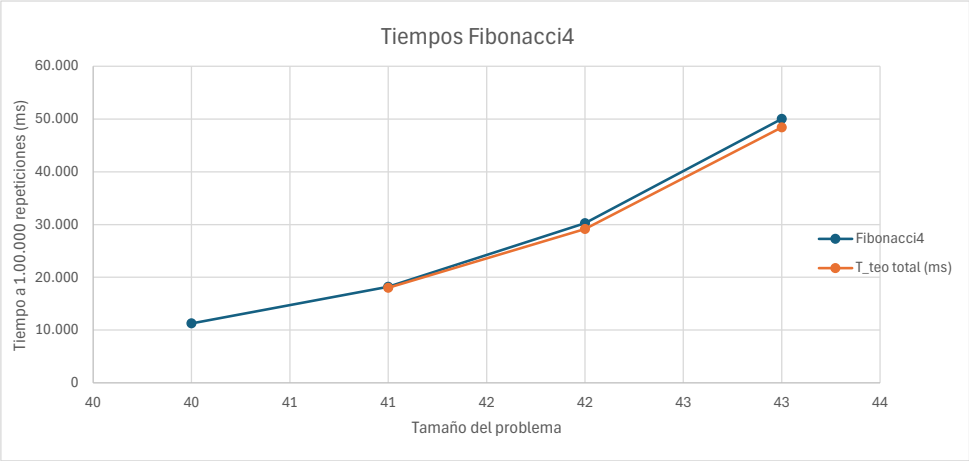
3.2. Gráficas de `Fibonacci`

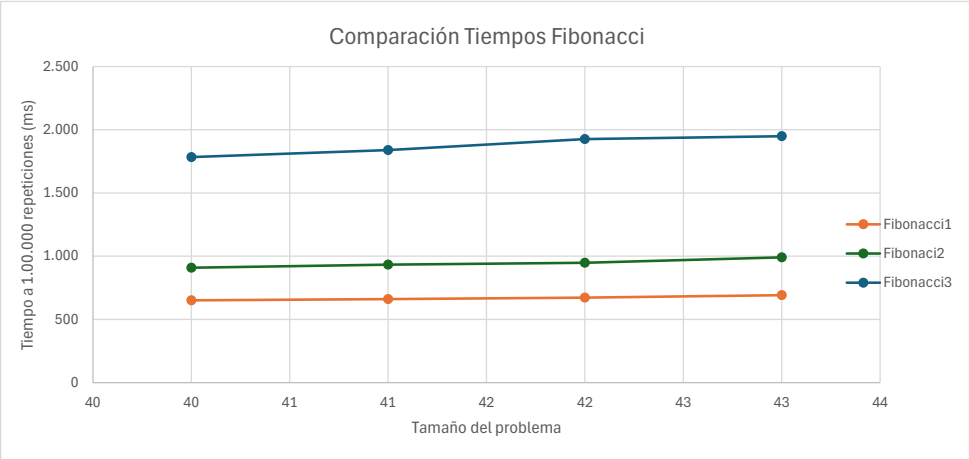
Hemos graficado el tiempo en milisegundos para 1.000.000 repeticiones de cada algoritmo (lo que equivale al tiempo promedio en nanosegundos para una repetición):

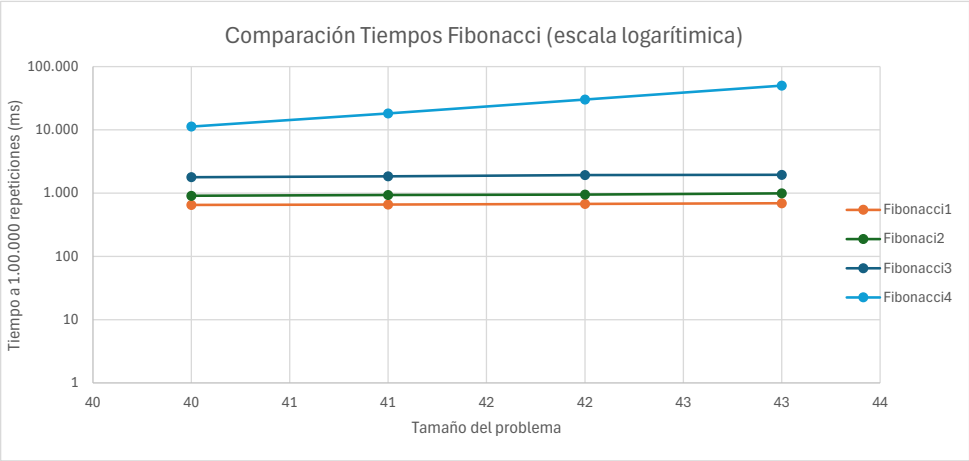





Tiempos Fibonacci3
Tiempo a 1.00.000 repeticiones (ms)
2.000
1.950
1.900
1.850
1.800
1.750
40
40
41
41
42
42
43
43
44
Tamaño del problema
Fibonacci3
T_teo total (ms)


Tiempos Fibonacci4
Tiempo a 1.00.000 repeticiones (ms)
60.000
50.000
40.000
30.000
20.000
10.000
0
40
40
41
41
42
42
43
43
44
Tamaño del problema
Fibonacci4
T_teo total (ms)





3.3. Tiempos de `VectorSuma`

Estos son los tiempos empíricos registrados, junto con la estimación teórica:

n	T_{total} (ms)	Repeticiones	n/T_{medio}	T_{teo} (ms)
1.000	227	1.000.000	4.405.286,3436	-
2.000	473	1.000.000	4.228.329,8097	454
4.000	938	1.000.000	4.264.392,3241	946

Método 1

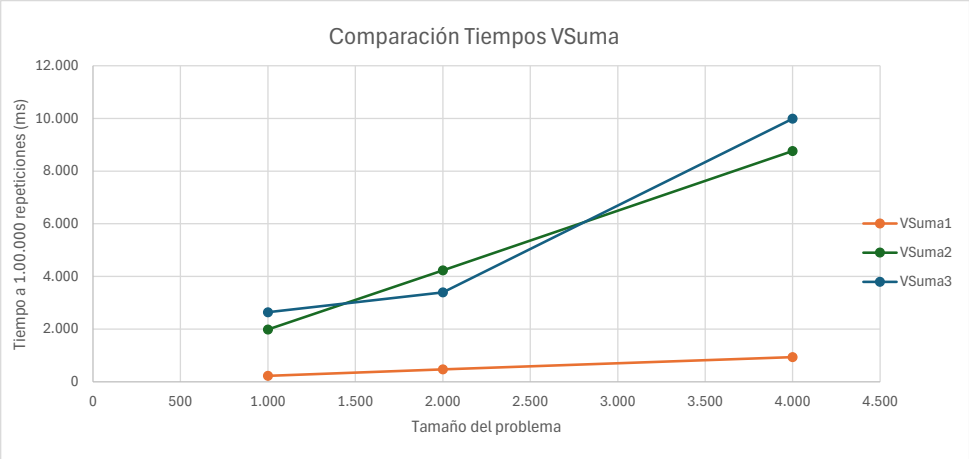
n	T_{total} (ms)	Repeticiones	n/T_{medio}	T_{teo} (ms)
1.000	1.987	1.000.000	503.271,2632	-
2.000	4.232	1.000.000	472.589,7921	3.974
4.000	8.762	1.000.000	456.516,7770	8.464

Método 2

n	T_{total} (ms)	Repeticiones	n/T_{medio}	T_{teo} (ms)
1.000	2.643	1.000.000	378.357,9266	-
2.000	3.395	1.000.000	589.101,6200	5.286
4.000	9.989	1.000.000	400.440,4845	6.790

Método 3

De nuevo, los algoritmos tienen todos una complejidad teórica idéntica, pero los tiempos reales registrados varían entre ellos. Esto evidencia que las clasificaciones como $O(n)$ agrupan funciones dispares. Por ejemplo, pueden verse involucradas las constantes multiplicativas que intervienen en la definición.



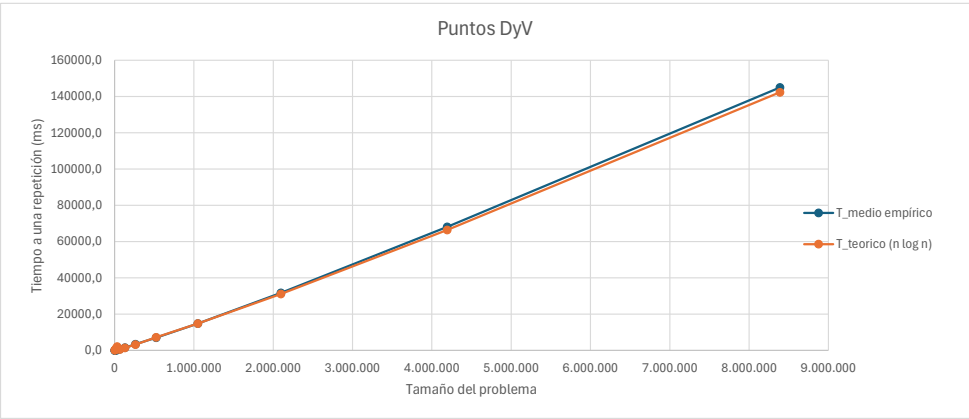
Se comparan los tiempos empíricos medidos para los tres algoritmos

§4. Puntos más cercanos de un conjunto

4.1. Ejecución secuencial

Tras clear una clase `PuntosDVTiempos`, estos han sido los resultados empíricos:

n	T_{total} (ms)	Repeticiones	$n\log(n)/T_{\text{medio}}$	n^2/T_{medio}	$T_{\text{teo}}(ms)$
1.024	56	10	550,4548	187.245,7143	-
2.048	100	10	678,1604	419.430,4000	123
4.096	204	10	725,3052	822.412,5490	218
8.192	439	10	730,2617	1.528.675,7175	442
16.384	972	10	710,3812	2.761.681,6461	946
32.768	2.131	10	694,3325	5.038.675,8517	2.083
65.536	666	1	473,9532	6.448.899,8438	455
131.072	1.512	1	443,6258	11.362.347,3439	1.415
262.144	3.378	1	420,4967	20.343.243,5571	3.202
524.288	7.001	1	428,3248	39.262.663,4686	7.131
1.048.576	14.790	1	426,8463	74.341.556,9828	14.739
2.097.152	31.699	1	418,2283	138.744.014,3570	31.059
4.194.304	68.074	1	408,0478	258.427.388,4951	66.417
8.388.608	144.977	1	400,6161	485.378.675,0841	142.337
16.777.216	FdT	-	-	-	-



Tiempos empírico y teórico promedio a una repetición

Hemos mejorado la complejidad que tenía la solución trivial ($O(n^2)$) al pasar a nuestro algoritmo de divide y vencerás. La complejidad para esta última versión es de $O(n \log n)$.

La implementación de `PuntosDV` del repositorio ya resuelve problemas de tamaño que no sea potencia de dos, por lo que no es necesario hacer una implementación por separado para cualquier tamaño $n \geq 3$. Sus resultados se han comprobado con los casos de prueba adecuadamente a través de una clase de casos de pruebas de `JUnit`.

Podemos observar cómo los tiempos teóricos se acercan mucho a los empíricos, tanto en la tabla como en la gráfica. Los cálculos de los tiempos teóricos se han llevado a cabo usando la complejidad $O(n \log n)$.

4.2. Paralelización del algoritmo

El algoritmo trabaja con Divide y Vencerás por división. Podemos asignar cada mitad a un nuevo hilo, siempre que la carga restante sea menor a un cierto umbral, que ajustaremos empíricamente para lograr un mejor rendimiento. De no trabajar con este umbral, el costo de gestionar los hilos sería mayor que los beneficios de la paralelización, dando lugar a peores resultados que la versión secuencial del algoritmo.

A nivel de implementación, usaremos el *framework* `ForkJoinPool` para lograr la paralelización. Crearemos una clase `PuntosDVParalelo` que herede de `Task<PuntosDTO>` para llevar la cuenta de los valores intermedios en cada paso del algoritmo. Una vez tengamos los resultados de ambos lados (gracias a la sincronización adecuada de los hilos), combinaremos la solución de cada uno como en el caso secuencial.

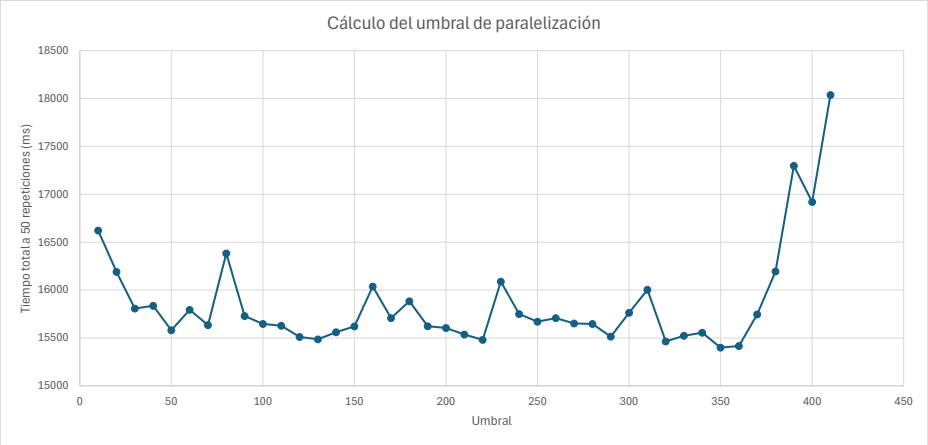
Con el fin de reducir la duplicidad del código, ciertos métodos estáticos de la clase secuencial original `PuntosDV` han pasado a tener ocultación de paquete. Esto no afecta al paralelismo ni al algoritmo original, simplemente tiene como fin un mantenimiento más fácil del código. Para garantizar el correcto funcionamiento del código, la nueva clase ha sido puesta a prueba con los correspondientes test de `JUnit`.

4.3. Cálculo del umbral

Para encontrar un buen umbral de paralelización, fijamos un tamaño de carga suficientemente grande, y medimos cuánto tarda neustro algoritmo para una serie de umbrales distintos. Con el fin de mitigar ruidos y azar, los tiempos medidos serán los totales para 50 repeticiones del algoritmo. En primer lugar, comencemos con una criba amplia:

n	Umbral	Tiempo total (ms)	Repeticiones
65536	10	16621	50
65536	20	16190	50
65536	30	15808	50
65536	40	15836	50
65536	50	15580	50
65536	60	15794	50
65536	70	15635	50
65536	80	16383	50
65536	90	15730	50
65536	100	15647	50
65536	110	15628	50
65536	120	15512	50
65536	130	15486	50
65536	140	15561	50
65536	150	15622	50
65536	160	16037	50
65536	170	15707	50
65536	180	15883	50
65536	190	15623	50
65536	200	15604	50
65536	210	15536	50
65536	220	15481	50
65536	230	16089	50
65536	240	15750	50

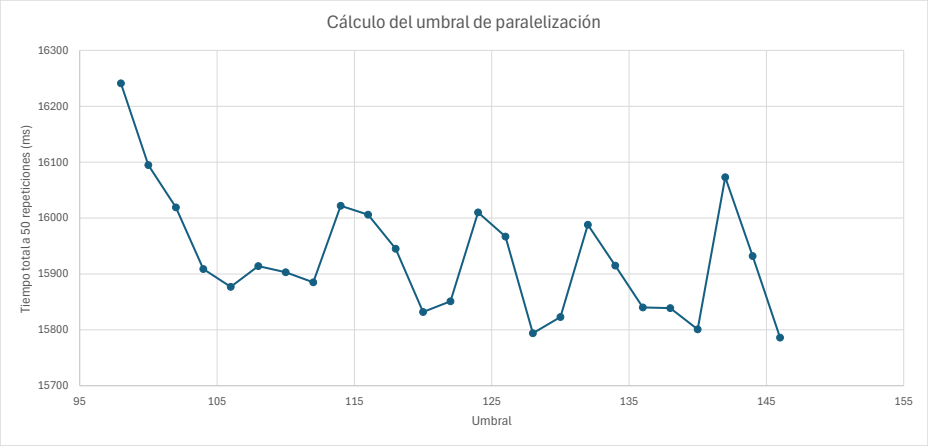
n	Umbral	Tiempo total (ms)	Repeticiones
65536	250	15670	50
65536	260	15707	50
65536	270	15652	50
65536	280	15647	50
65536	290	15515	50
65536	300	15764	50
65536	310	16003	50
65536	320	15463	50
65536	330	15523	50
65536	340	15555	50
65536	350	15399	50
65536	360	15417	50
65536	370	15747	50
65536	380	16195	50
65536	390	17298	50
65536	400	16920	50



Tiempos que tarda el algoritmo para los distintos umbrales considerados

A pesar de no haber un codo claro, podemos apreciar cómo la región de umbral entre 100 y 150 posee tanto tiempos relativamente bajos como estabilidad. Exploremos más valores concretos dentro de este área a modo de "zoom".

n	Umbral	Tiempo total (ms)	Repeticiones
65536	98	16241	50
65536	100	16095	50
65536	102	16019	50
65536	104	15909	50
65536	106	15877	50
65536	108	15914	50
65536	110	15903	50
65536	112	15885	50
65536	114	16022	50
65536	116	16006	50
65536	118	15945	50
65536	120	15832	50
65536	122	15851	50
65536	124	16010	50
65536	126	15967	50
65536	128	15794	50
65536	130	15823	50
65536	132	15988	50
65536	134	15915	50
65536	136	15840	50
65536	138	15839	50
65536	140	15801	50
65536	142	16073	50
65536	144	15932	50
65536	146	15786	50
65536	148	16013	50
65536	150	15970	50
65536	152	16074	50
65536	154	16051	50



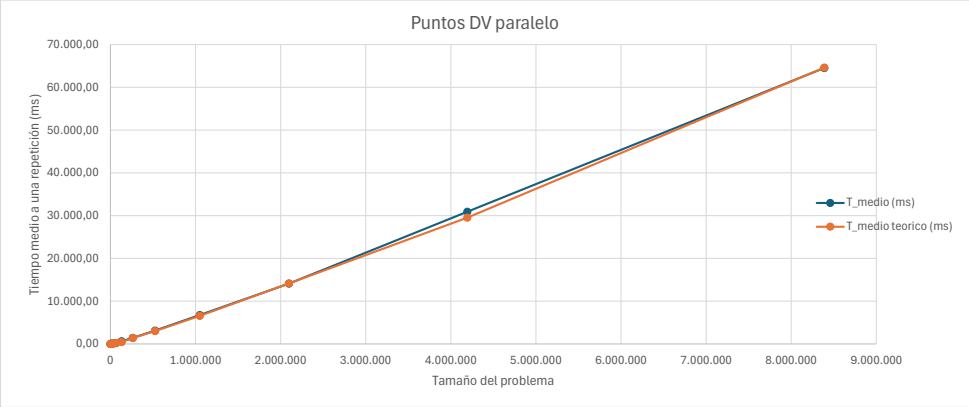
Los errores relativos son muy pequeños. No hay una región de mejoría clara. En base a las mediciones realizadas, un umbral de 108 presenta tiempos aceptables, así como robustez y estabilidad.

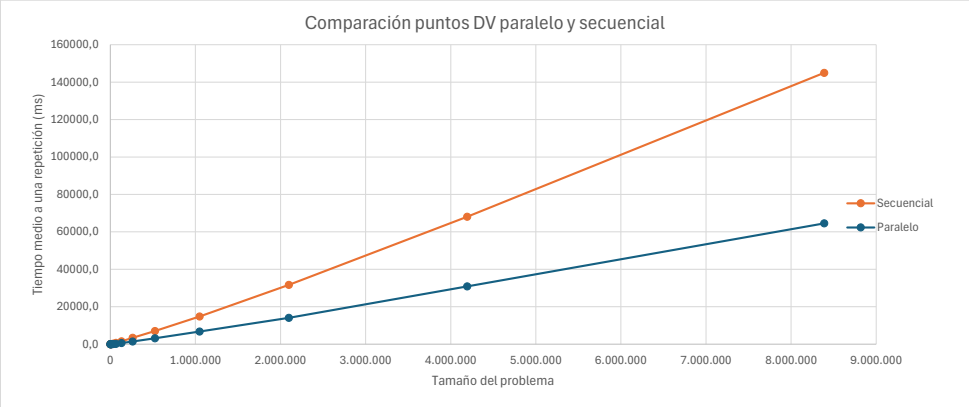
Nota: la clase paralelizada tiene un umbral de 108 por las mediciones anteriores. No obstante, se mantiene la funcionalidad de cambio de umbral con nivel de ocultación `package` para que los resultados puedan ser replicados. Esta funcionalidad es solo de prueba y no se vería incluida en una versión "de producción".

4.4. Comparación con la versión secuencial

Ahora que hemos determinado un buen umbral para nuestro algoritmo, veamos qué tan notoria es la diferencia en comparación con la versión secuencial:

n	T (ms)	Repeticiones	T_{medio} (ms)	T_{medio} teórico (ms)
1.024	205	100	2,05	-
2.048	335	100	3,35	4,5100
4.096	657	100	6,57	7,3091
8.192	1.314	100	13,14	14,2350
16.384	2.525	100	25,25	28,3015
32.768	10.643	100	106,43	54,1071
65.536	20.031	100	200,31	227,0507
131.072	644	1	644,00	425,6588
262.144	1.424	1	1.424,00	1.363,7647
524.288	3.112	1	3.112,00	3.006,2222
1.048.576	6.743	1	6.743,00	6.551,5789
2.097.152	14.097	1	14.097,00	14.160,3000
4.194.304	30.901	1	30.901,00	29.536,5714
8.388.608	64.531	1	64.531,00	64.611,1818
16.777.216	FdT	-	-	-





Aunque la complejidad computacional teórica sigue siendo la misma, podemos observar una clara mejora en los tiempos medidos.

§5. Condiciones de medida

5.1. Condiciones para la práctica

Con respecto a las secciones 1 a 3 de esta práctica, todas las medias han sido realizadas en Eclipse IDE con la opción `-Xint` de la máquina virtual, y con el siguiente hardware:

```
CPU 12th Gen Intel(R) Core(TM) i5-12400 @ 2,5GHz + 16GB RAM
```

La sección 4 ha sido medida también en Eclipse, pero con la optimización de Java. Además, se ha usado el siguiente hardware:

```
CPU: Intel Core i7-6700HQ CPU @ 2,60GHz + 16GB RAM
```